Análisis de sentimientos en español mediante un Transformer preentrenado

Salvador López Trigueros
Curso de Especialización en Inteligencia Artificial y Big Data
Modelos de Inteligencia Artificial
Actividad NF2

Centro educativo: Planeta FP

1 Introducción **3**
2 Fundamentos del procesamiento del lenguaje natural **3**
3 Aplicaciones y ventajas **4**
4 Modelo seleccionado **4**
5 Diseño de la implementación **4**
6 Diseño de las pruebas **5**
7 Evaluación de la calidad **5**
8 Limitaciones y sesgos **6**
9 Seguridad y uso responsable **7**
10 Conclusiones y posibles mejoras **7**
11 Declaración de uso de inteligencia artificial generativa **8**
12 Bibliografía **8**

1 Introducción

El procesamiento del lenguaje natural (PLN) es el ámbito de la inteligencia artificial que desarrolla métodos para analizar y procesar lenguaje humano. Sus aplicaciones incluyen la clasificación de documentos, la traducción, la extracción de información y los asistentes conversacionales. Una de sus tareas es el análisis de sentimientos, que intenta identificar la orientación de una opinión expresada en un texto.

El objetivo de este proyecto es diseñar una pequeña aplicación en Python que clasifique comentarios en español sobre una aplicación informática como positivos, negativos o neutrales. Para ello se integrará un Transformer preentrenado y ajustado previamente para esta tarea. No se realizará un entrenamiento desde cero ni un nuevo ajuste de sus parámetros.

2 Fundamentos del procesamiento del lenguaje natural

El texto debe convertirse en una representación numérica antes de introducirlo en una red neuronal. El tokenizador divide la entrada en unidades llamadas tokens, que pueden ser palabras, fragmentos de palabras o signos. A continuación, les asigna identificadores numéricos. Por ello, una palabra no equivale necesariamente a un token.

Las representaciones vectoriales permiten procesar esas unidades mediante operaciones matemáticas. En un Transformer se utiliza atención para combinar información de distintas posiciones de la secuencia. Esto ayuda a construir representaciones que dependen del contexto: una palabra puede adquirir un significado distinto según el resto de la frase.

El preentrenamiento permite aprender patrones lingüísticos a partir de grandes colecciones de texto. Posteriormente, el ajuste para una tarea, o fine-tuning, adapta los parámetros utilizando ejemplos etiquetados. En este proyecto ambos procesos fueron realizados por los autores del modelo; la implementación utiliza el resultado para obtener predicciones sobre nuevas entradas.

Un Transformer es una arquitectura de red neuronal. Un chatbot, en cambio, es una aplicación conversacional que puede construirse con reglas o con modelos aprendidos. El sistema de esta actividad es un clasificador: selecciona una etiqueta y no genera una respuesta conversacional ni texto libre.

Enfoque	Funcionamiento	Ventajas y límites
Reglas	Patrones definidos manualmente.	Fáciles de inspeccionar; cobertura limitada a las expresiones previstas.
Redes neuronales	Aprenden parámetros a partir de datos.	Pueden generalizar; dependen de los datos y requieren evaluación.
Transformers	Redes con mecanismos de atención.	Representan contexto; pueden fallar fuera de su dominio y requieren más recursos.

3 Aplicaciones y ventajas

Un clasificador de sentimientos puede apoyar la exploración de reseñas, organizar comentarios para revisión o mostrar tendencias en opiniones. En una aplicación informática permitiría obtener una primera agrupación de mensajes favorables, desfavorables y descriptivos. El sentimiento no equivale a urgencia: un mensaje neutral puede describir un problema crítico.

Reutilizar un modelo ajustado reduce el trabajo de entrenamiento y permite centrarse en la integración y evaluación. También facilita procesar conjuntos de textos de forma consistente. Estas ventajas no garantizan precisión en todos los contextos: es necesario comprobar su comportamiento en el dominio de uso y revisar las decisiones relevantes.

4 Modelo seleccionado

Se ha utilizado pysentimiento/robertuito-sentiment-analysis, publicado en Hugging Face. Su ficha indica que está basado en RoBERTuito y ajustado con aproximadamente 5.000 tuits del corpus TASS 2020 de distintas variedades del español. El modelo base se describe en el artículo de Pérez y colaboradores.

Las etiquetas verificadas en su configuración son NEG, NEU y POS, correspondientes a negativo, neutral y positivo. La neutralidad se interpreta aquí como ausencia de una valoración positiva o negativa clara. Las opiniones mixtas pueden resultar difíciles de clasificar porque el modelo devuelve una única etiqueta para todo el comentario. En esta implementación no se realiza un análisis separado de esos casos.

La elección se justifica por su orientación al español y al texto breve de redes sociales. Los comentarios de una aplicación pueden compartir rasgos informales aunque la semántica y composición pueden ser diferentes.

La implementación utiliza la función pipeline de Hugging Face Transformers para cargar el modelo y realizar las predicciones. Se ejecuta en CPU mediante el parámetro device=-1. La primera descarga requiere conexión a internet y espacio para almacenar los archivos del modelo. En esta actividad no se han medido los tiempos de ejecución ni el consumo de memoria.

5 Diseño de la implementación

La implementación se ha desarrollado en un notebook de Python. Se utiliza la función pipeline de Hugging Face Transformers con la tarea sentiment-analysis y el modelo pysentimiento/robertuito-sentiment-analysis.

Los comentarios y sus etiquetas de referencia se almacenan en un DataFrame de pandas. A continuación, se proporciona la lista de textos al clasificador, que se encarga de tokenizarlos, ejecutar el modelo y devolver una etiqueta y una puntuación para cada comentario.

Se utilizan los parámetros truncation=True y max_length=128 para limitar las entradas a 128 tokens. Los textos que superen ese límite se recortarán, aunque esta implementación no muestra una advertencia específica. No se ha incorporado una comprobación explícita de entradas vacías, ya que los ejemplos utilizados están definidos previamente y contienen texto.

Las predicciones se añaden al DataFrame junto con sus puntuaciones. También se crea la columna acierto, que indica si la predicción coincide con la etiqueta de referencia.

La puntuación asociada a cada predicción expresa la salida del modelo para la clase seleccionada. Una puntuación elevada no garantiza que la clasificación sea correcta ni demuestra que el modelo esté calibrado.

Los resultados se muestran directamente en el notebook.

6 Diseño de las pruebas

Se ha utilizado un conjunto de nueve comentarios ficticios relacionados con una aplicación informática. Los ejemplos se distribuyen de forma equilibrada: tres positivos, tres negativos y tres neutrales.

Cada comentario tiene una etiqueta de referencia que permite comparar el resultado esperado con la predicción del modelo. Los textos se han propuesto con ayuda de IA generativa y constituyen una demostración exploratoria, no un conjunto representativo de comentarios reales.

Los ejemplos incluyen valoraciones positivas, quejas y mensajes descriptivos sin una valoración clara. No se han realizado pruebas específicas de ironía, opiniones mixtas, entradas vacías o textos extensos.

No se ha dividido este conjunto en entrenamiento y test porque se utiliza un modelo preentrenado cuyos parámetros no se modifican. Los nueve comentarios se emplean únicamente para comprobar y evaluar su comportamiento en esta demostración.

7 Evaluación de la calidad

La evaluación se ha realizado comparando las etiquetas predichas con las etiquetas de referencia de los nueve comentarios. Se ha utilizado classification_report de scikit-learn para obtener accuracy, precision, recall y F1-score.

Precision indica qué proporción de las predicciones de una clase coincide con la referencia. Recall indica qué proporción de los ejemplos de esa clase se detecta correctamente. F1 combina ambas medidas mediante su media armónica. F1 macro calcula el promedio del F1 de las tres clases, dando el mismo peso a cada una.

El modelo ha clasificado correctamente los nueve comentarios y ha obtenido los siguientes resultados:

Métrica	Resultado
Accuracy	1,00
Precision por clase	1,00 en NEG, NEU y POS
Recall por clase	1,00 en NEG, NEU y POS
F1-score por clase	1,00 en NEG, NEU y POS
F1 macro	1,00

La matriz de confusión presenta tres aciertos por categoría y ningún error. Sus filas representan las etiquetas de referencia y sus columnas las predicciones.

La última celda del notebook muestra únicamente las filas clasificadas incorrectamente. En esta ejecución devuelve un DataFrame vacío porque todas las predicciones coinciden con sus etiquetas de referencia.

Los resultados son perfectos, pero hay que tener en cuenta que la muestra es muy pequeña y que estos resultados pueden estar muy inflados y no garantizan que el modelo tenga el mismo rendimiento frente a otros datos.

8 Limitaciones y sesgos

La ironía puede invertir el sentido literal de una frase. Las negaciones y las opiniones mixtas también dificultan asignar una única etiqueta. Un mensaje como «La interfaz es bonita, pero pierdo mis archivos» contiene valoraciones diferentes y exige considerar el aspecto al que se refiere cada una.

Las variantes regionales, los errores ortográficos, los emojis y las expresiones de grupos poco representados pueden producir diferencias de rendimiento. La presencia de varias variedades del español en el entrenamiento no demuestra que todas estén igualmente representadas o que el modelo funcione igual en cada una.

Los textos largos pueden perder información al truncarse. Además, las etiquetas humanas son interpretaciones: distintos anotadores pueden discrepar.

El conjunto utilizado es pequeño, está equilibrado y contiene comentarios diseñados manualmente con ayuda de IA generativa. Puede resultar más sencillo que los textos de usuarios

reales y reflejar los criterios de quien lo ha preparado. Aunque no se han observado errores en estos nueve ejemplos, no se han comprobado sistemáticamente las limitaciones mencionadas. La ausencia de fallos en esta demostración no demuestra robustez frente a ironía, variaciones dialectales o comentarios extensos.

9 Seguridad y uso responsable

Las pruebas utilizan comentarios ficticios sin datos personales. La ejecución propuesta realiza inferencia local con los pesos descargados; no llama a un servicio remoto de predicción. Aun así, el entorno y la descarga pueden generar comunicaciones de red, por lo que no se debe afirmar que todo el proceso está completamente desconectado.

Los comentarios se procesan como texto y no se ejecutan como instrucciones. La implementación utiliza la función estándar pipeline de Transformers y no solicita ejecutar código remoto personalizado. Las etiquetas obtenidas se muestran únicamente con fines educativos y no desencadenan decisiones automáticas sobre usuarios. Las pruebas emplean comentarios ficticios y los resultados se presentan dentro del notebook, sin implementar una exportación a archivos.

Este clasificador produce etiquetas de sentimiento, por lo que no tiene el mismo funcionamiento que un chatbot generativo. Sus errores principales son clasificaciones incorrectas y puntuaciones engañosamente altas, no la invención de una respuesta textual extensa.

10 Conclusiones y posibles mejoras

La actividad ha permitido integrar un Transformer preentrenado en un notebook de Python y utilizarlo para clasificar sentimientos en comentarios en español. La función pipeline simplifica la tokenización y la ejecución del modelo, permitiendo centrarse en las predicciones y su evaluación.

En los nueve ejemplos evaluados, el modelo ha obtenido un accuracy y un F1 macro de 1,00. La matriz de confusión confirma que los tres comentarios de cada categoría se han clasificado correctamente.

Estos resultados son satisfactorios como demostración del funcionamiento de la implementación, pero no permiten asegurar el mismo rendimiento en situaciones reales debido al tamaño y la sencillez del conjunto utilizado.

Como posibles mejoras se propone ampliar las pruebas con comentarios representativos del ámbito de uso, incorporar ejemplos de ironía y opiniones mixtas, y revisar las etiquetas con otra persona. También podría añadirse una gestión explícita de entradas vacías y una advertencia cuando los textos superen el límite de tokens.

11 Declaración de uso de inteligencia artificial generativa

Se ha utilizado ChatGPT 6.1 Sol como apoyo para estructurar y redactar este documento, proponer código y elaborar ejemplos ficticios de evaluación.

12 Bibliografía

[1] Jurafsky, D. y Martin, J. H. (2026). Speech and Language Processing, tercera edición en borrador, versión del 19 de agosto de 2026. https://web.stanford.edu/~jurafsky/slp3/ . Referencia actual del periodo 2024–2026 para fundamentos de PLN y Transformers.

[2] Hugging Face (s. f.). Text classification. Documentación de Transformers. https://huggingface.co/docs/transformers/en/tasks/sequence_classification . Consulta: 28 de octubre de 2026.

[3] pysentimiento (s. f.). robertuito-sentiment-analysis. Ficha del modelo. https://huggingface.co/pysentimiento/robertuito-sentiment-analysis . Consulta: 28 de octubre de 2026.

[4] Pérez, J. M., Furman, D. A., Alonso Alemany, L. y Luque, F. M. (2022). RoBERTuito: a pre-trained language model for social media text in Spanish. Proceedings of LREC 2022, 7235–7243. https://aclanthology.org/2022.lrec-1.785/

[5] pysentimiento (s. f.). Configuración de robertuito-sentiment-analysis. https://huggingface.co/pysentimiento/robertuito-sentiment-analysis/blob/main/config.json . Consulta: 28 de octubre de 2026.

[6] pysentimiento (s. f.). Repositorio y apartado License. https://github.com/pysentimiento/pysentimiento . Consulta: 28 de octubre de 2026.

[7] scikit-learn (s. f.). classification_report. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html . Consulta: 28 de octubre de 2026.

Link Video Implementación : https://youtu.be/RFlJfpfq6A8